arXiv:2509.24517v3 [cs.LG] 13 Aug 2026

Performance-Carbon Trade-Offs across Architectural Biases in Shear Flow Forecasting

Sophia N. Wilson[1], Jens Hesselbjerg Christensen[2], and Raghavendra Selvan[1]

[1] Department of Computer Science, University of Copenhagen, Denmark
{sophia.wilson,raghav}@di.ku.dk
[2] Niels Bohr Institute, University of Copenhagen, Denmark
hesselbjerg@nbi.ku.dk

Abstract. Development of modern deep learning methods has been driven primarily by the push for improving model efficacy (accuracy metrics), leading to large-scale models that require massive computational resources and result in considerable carbon footprint across the model lifecycle. In this work, we explore how architectural biases, specifically a model's receptive field and periodicity assumption, are associated with the trade-offs between predictive performance and carbon footprint for spatio-temporal forecasting of incompressible shear flow. We study seven models differing in these properties and evaluate pointwise accuracy, physics-fidelity, and carbon cost across training and inference. We find that no single model dominates across both predictive performance and carbon cost, that a lower training cost does not straightforwardly extend to inference, and that runtime alone is an unreliable proxy for carbon cost, particularly during training. Together, these results underscore the importance of explicitly evaluating carbon costs across the full model lifecycle. We argue that model efficiency, alongside efficacy, should be a core consideration in machine learning model development and deployment.

Keywords: Resource-aware machine learning · Green AI · Inductive biases · Spatio-temporal forecasting

1 Introduction

Machine learning (ML) has advanced rapidly over the past decade, driven largely by the pursuit of improving model efficacy (accuracy metrics). This narrow focus has led to increasingly large models that require substantial computational resources and generate significant carbon emissions across their lifecycle [1, 20, 27, 28]. As models scale, efficacy gains diminish while costs escalate, underscoring the need to move beyond accuracy as the sole measure. A more holistic assessment must balance efficacy with efficiency, capturing not only predictive performance but also compute, energy, and carbon costs [10]. This requires identifying strategies that strike a favourable balance rather than optimising for accuracy alone.

Incorporating inductive biases into model design offers a promising path towards this balance. It is commonly understood in ML that strong model assumptions (bias) risk under-fitting, whereas high sensitivity to training data can lead to over-fitting (variance), and that striking the right balance is crucial to obtain the right class of models [15]. We draw an analogy to model design under resource constraints: a stronger inductive bias can substitute for some of what would otherwise need to be learned from data, potentially reducing the data and compute required to reach a given level of performance. This suggests that inductive bias may trade off not only against variance, but also against the resource cost associated with developing a model.

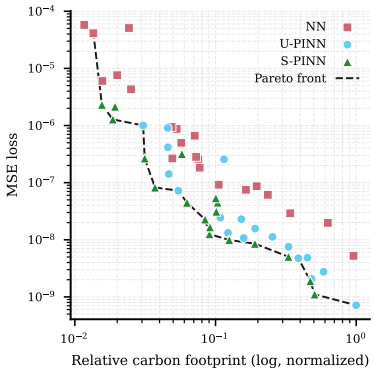


Fig. 1: PINN toy experiment. Test mean squared error (MSE) versus CO_2eq emissions for a data-driven NN (pink squares), an unsupervised PINN (blue dots), and a semi-supervised PINN (green triangles). The dashed line marks the Pareto front, dominated by the S-PINN, indicating the best accuracy-carbon trade-off. Data and code are adapted from [14].

In this work, we consider inductive biases as an umbrella term spanning techniques such as Bayesian priors over model weights, task-specific regularisation terms, and architectural biases embedded in model design. Prior work has shown that embedding conservation laws, symmetries, or governing equations can reduce the number of trainable parameters [8, 23], reduce the amount of training data [6, 24, 33], and accelerate training convergence [4, 12]. Whether inductive biases can also reduce energy consumption and carbon footprint remains less explored.

Fig. 1 offers a concrete illustration. In a simple experiment on a partial differential equation (PDE) system, we compare three models that share the same underlying architecture but differ in the strength of inductive bias imposed through their loss functions: a purely data-driven neural network (NN), an unsupervised physics-informed neural network (PINN) trained solely on PDE residuals, and a semi-supervised PINN combining data and PDE residuals. The semi-supervised PINN dominates the Pareto front, achieving lower prediction error at a lower carbon cost than either alternative. This suggests that striking the right balance between data and inductive bias may simultaneously improve accuracy and reduce the carbon footprint.

Building on this intuition, we extend the perspective from task-specific models such as PINNs to architectures that embed inductive biases structurally. We study this in the context of spatio-temporal forecasting of PDE dynamics, specifically of a complex, high-resolution simulation of 2D incompressible shear flow.

Our key contributions towards this end are:

1. **Carbon-aware evaluation:** We promote evaluation practices that explicitly account for training and inference carbon costs alongside predictive accuracy and physics-fidelity.

2. **Characterising the performance-carbon trade-off:** We provide an empirical characterisation of how architectural biases in receptive field and periodicity are associated with predictive performance and carbon footprint in spatio-temporal forecasting of incompressible shear flow.

2 Background and Related Work

The Question of Resource-Awareness. The standard metrics used to characterise the performance of ML models primarily focus on efficacy and are untethered from efficiency considerations. This introduces a preference for models that offer high efficacy at large resource costs [2]. There are two primary streams of work that aim to include resource-aware metrics: composite metrics and multi-objective optimisation.

Several composite metrics have been proposed to combine predictive performance with resources like energy consumption or carbon footprint. Evchenko *et al.* [9] introduced a composite metric that combines efficacy and efficiency metrics to study the influence of resource constraints on traditional (non deep learning) ML models. *Carburacy* [21] extends the idea to transformer models by jointly quantifying accuracy and carbon footprint. More recently, Kapoor *et al.* [13] proposed *EcoL2*, a composite metric similar to *Carburacy* but extended to include the costs of data generation and hyperparameter tuning.

Multi-objective optimisation has been primarily used in neural architecture search (NAS), resulting in models like EfficientNet [29]. More recently, energy consumption [2] and carbon footprint [32] have been taken into consideration in NAS. These methods, however, have not studied how architectural biases relate to the resource consumption of ML models.

Spatio-Temporal Forecasting Models and Architectural Biases. There are many model classes for the task of spatio-temporal forecasting, differing in the architectural assumptions they encode. In this work, we focus on two: the UNet [26] and the Fourier Neural Operator (FNO) [17], both originally developed for image-like data, as used in this study (Sec. 3).

UNets perform forecasting by mapping a window of past states to future states, using convolutional layers that are inherently biased towards locality (each layer only mixes information within a limited receptive field) and translation equivariance (the same filters are applied everywhere in the domain). Skip connections fuse features across multiple spatial scales, but the model does not impose explicit constraints on the underlying dynamics beyond what is learned from data. By default, convolutions pad the input with zeros at the domain boundary, which implicitly assumes a non-periodic boundary, inconsistent with

the periodic domain we study (Sec. 3). Circular padding corrects this inconsistency by replacing zero-padding with a periodic boundary assumption.

The FNO instead represents the state in Fourier space, letting the model mix information globally in a single layer rather than through stacked local convolutions. Like the UNet, it is translation-equivariant, since multiplication in Fourier space corresponds to a spatial convolution. In contrast to the UNet, however, its receptive field is global rather than local, and its use of the discrete Fourier transform assumes periodicity throughout the representation, rather than only at the domain boundary as with circular padding. Truncating to a finite number of Fourier modes additionally biases the model towards smooth, low-frequency solutions. We adapt the FNO from its original operator-learning formulation [17] to an autoregressive setup.

3 Experiment

Problem Setup. We consider spatio-temporal forecasting of dynamical systems, where the objective is to approximate the solution $u(x, y, t)$ of a PDE given initial and boundary conditions. We frame this as an autoregressive prediction on a two-dimensional, uniformly spaced mesh $(x, y) \in \{x_1, x_2, \ldots, x_H\} \times \{y_1, y_2, \ldots, y_W\}$ with discretised time steps $t \in \{t_1, t_2, \ldots, t_T\}$. At each step t_k the state is represented as a tensor $u_k(x, y) \in \mathbb{R}^{H \times W \times C}$, where $k \in \{1, 2, \ldots, T\}$ indexes discrete time steps in a trajectory of length T. H and W are spatial dimensions and C is the number of physical fields (channels), such as pressure or velocity components.

To capture temporal dependencies, the model is conditioned on a finite history of length h. The input window is denoted as $u_{k-h+1:k}(x, y) := (u_{k-h+1}(x, y), \ldots, u_k(x.y))$. A parameterised function f_θ is trained to predict:

$$u_{k+1}(x, y) = f_\theta\big(u_{k-h+1:k}(x, y)\big). \tag{1}$$

During training, the loss is computed on one-step predictions. At inference, we assess long-term stability via autoregressive rollouts, obtained by iteratively applying Eq. 1 and feeding the models own predictions back as inputs.

We instantiate this framework for forecasting incompressible shear flow governed by the Navier–Stokes equations, a setting that is both computationally demanding and governed by well-understood physical structure, making it well-suited for studying the efficacy-efficiency trade-off.

Data. We use the incompressible shear flow data from *The Well* collection [22], generated with *Dedalus* [5]. The data describes 2D incompressible Navier–Stokes dynamics with an additional passive tracer,

$$\partial_t \mathbf{u} + \nabla p - \nu \nabla^2 \mathbf{u} = -\mathbf{u} \cdot \nabla \mathbf{u}, \tag{2}$$

$$\partial_t s - D \nabla^2 s = -\mathbf{u} \cdot \nabla s, \tag{3}$$

$$\nabla \cdot \mathbf{u} = 0, \tag{4}$$

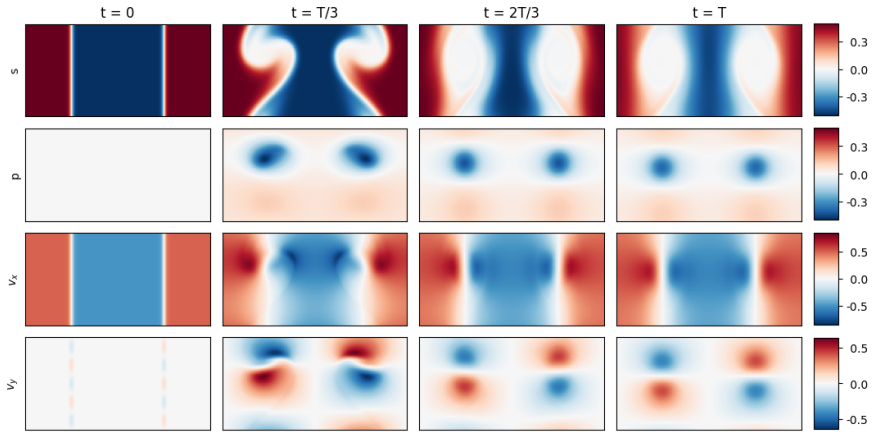


Fig. 2: Example of the incompressible shear flow data. Temporal evolution of tracer (s), pressure (p), horizontal velocity (v_x) and vertical velocity (v_y) fields at four time steps: $t \in \{0, T/3, 2T/3, T\}$. The snapshots illustrate the coupled dynamics of the fields.

where $\mathbf{u} = (u_x, u_y)$ are the velocity components, p is pressure satisfying $\int p = 0$, s is the passive tracer, ν is viscosity, and D is diffusivity, related to Reynolds and Schmidt numbers as $\nu = 1/\mathrm{Re}$ and $D = \nu/\mathrm{Sc}$. The dataset captures the coupled non-linear evolution of these four fields on a periodic spatial domain (see example in Fig. 2). The system is initialised with a horizontal shear flow perturbed by small sinusoidal variations in the vertical velocity, localised at the shear layers; the tracer is initialised to match the shear flow and pressure to zero. Initial conditions are governed by three parameters: the number of shear layers $n_{\mathrm{shear}} \in \{2, 4\}$, the number of perturbation oscillations $n_{\mathrm{blobs}} \in \{2, 3, 4, 5\}$, and a width parameter $w \in \{0.25, 0.5, 1.0, 2.0, 4.0\}$ controlling the sharpness of the shear transition and localisation of the perturbations. Each field is standardised as $\hat{x} = (x - \mu)/\sigma$ prior to training.

We work with a subset, which consists of 240 simulations spanning six PDE parameter settings and 40 initial conditions settings. Simulations are resolved at 256×512 with 200 time steps ($\Delta t = 0.1$). The data is split by initial condition into 80/10/10 for training, validation, and testing, yielding $37,632$ one-step samples for training, $4,704$ for one-step inference, and 24 trajectories for autoregressive rollout initialised at $t = 0$.

Models. We evaluate seven models that vary in receptive field and periodicity assumption, summarised in Table 1. We consider two convolutional architectures with a local receptive field and no periodicity assumption: UNet and a ConvNeXt variant (UNet-CN) [18], where the latter improves hardware efficiency through modern convolutional design. We introduce an explicit periodicity assumption by applying circular padding to both of these architectures (UNet-CP, UNet-CN-CP), while keeping their local receptive field unchanged. The remaining three models instead combine a global receptive field with an implicit period-

Table 1: Architectural biases.

Model	**Receptive field**	**Periodicity**
UNet, UNet-CN	Local	None (zero-padded)
UNet-CP, UNet-CN-CP	Local	Explicit (circular padding)
FNO, TFNO, UFNO	Global	Implicit (Fourier basis)

Table 2: Model configurations.

Model	**Filter size/modes**	**Init dim**	**Blocks/stage**	**Up/down**	**Bottleneck**
UNet, UNet-CP	3	48	1	4	1
UNet-CN, UNet-CN-CP	7	42	2	4	1
FNO, TFNO	16	128	4	–	–
UFNO	8, 12, 16	128	1	3	1

icity assumption, achieved through a Fourier representation: a baseline FNO [17], a Tucker-factorised variant (TFNO) [16] that improves parameter efficiency, and a U-shaped variant (UFNO*) [25] designed to better capture multi-scale interactions. Model configurations follow prior work: UNets, FNO, and TFNO follow Ohana *et al.* [22], while UFNO follows Rahman *et al.* [25]. All models are scaled to a comparable size: 17.5M parameters for UNet and UNet-CN, 18.6M for UNet-CP and UNet-CN-CP, and 19.0M, 19.3M, and 18.8M for FNO, TFNO, and UFNO, respectively. Table 2 summarises the full model configurations.

Training Procedure. All models use a history of $h = 4$ states and are trained for one-step prediction according to Eq. 1. Across all models, we use a batch size of 16 and AdamW optimiser [19] with 10^{-4} weight decay. We adopt the coarse-tuned learning rates reported in *The Well* [22] ($5 \cdot 10^{-4}$ for UNets and 10^{-3} for FNOs, respectively), which are tuned across several datasets including the one used here. A linear warm-up cosine scheduler is applied for the first three epochs and the loss is the mean squared error (MSE) averaged across fields and space. Training run on a single Nvidia A40 GPU ($\leq$ 24h) with early stopping (patience 6).

Evaluation Metrics. We assess forecasting performance using four metrics: two accuracy metrics, variance-scaled root mean squared error (VRMSE) and Pearson correlation coefficient (Pearson r), and two physics-fidelity metrics, velocity divergence and spectral energy error. VRMSE and Pearson r are computed on all four physical fields (tracer, pressure, and both velocity components) and averaged across fields and trajectories. Velocity divergence and spectral energy error are computed only on the velocity field $\mathbf{u} = (u_x, u_y)$, since incompressibility and kinetic energy are properties of the velocity rather than of the tracer or pressure.

VRMSE reports the root mean squared error between a predicted field $\hat{y}$ and its ground truth y, normalised by the spatial variance of the ground truth.

* We denote this as UFNO for consistency with our FNO/TFNO naming, but it is not to be confused with the distinct U-FNO architecture by Wen *et al.* [30].

This normalisation accounts for the fact that the four fields differ substantially in their natural variability. A VRMSE greater than one indicates that the prediction error exceeds the field's own fluctuations. Pearson r quantifies the linear correlation between the prediction and ground truth. Unlike VRMSE, it is invariant to differences in scale and variance across fields, so a high r can coexist with a systematically biased or miscalibrated prediction, which motivates the two physics-fidelity metrics below.

Velocity divergence quantifies violation of the incompressibility constraint (Eq. 4) in the predicted velocity field. We compute the divergence $\nabla \cdot \mathbf{u}$ via periodic central differences and summarise it at each rollout step as the spatial root mean square, averaged over trajectories,

$$\mathrm{Div}(\mathbf{u}) = \left\langle \left(\langle (\nabla \cdot \mathbf{u})^2 \rangle \right)^{1/2} \right\rangle_{\text{trajectories}}, \tag{5}$$

where $\langle \cdot \rangle$ denotes spatial averaging. Spectral energy error measures whether a model preserves the correct distribution of kinetic energy across spatial scales. We compute the radially-binned kinetic energy spectrum $E(\mathbf{u}; k)$ of the velocity field via its 2D discrete Fourier transform, binning by wavenumber $k = \sqrt{k_x^2 + k_y^2}$ into $N_{\text{bins}} = \lfloor \min(H, W)/2 \rfloor$ shells, and report the relative ℓ_2 error between the predicted and reference spectra, averaged over trajectories,

$$\mathrm{SpecErr}(\hat{\mathbf{u}}, \mathbf{u}) = \left\langle \frac{\| E(\hat{\mathbf{u}}; \cdot) - E(\mathbf{u}; \cdot) \|_2}{\| E(\mathbf{u}; \cdot) \|_2 + \epsilon} \right\rangle_{\text{trajectories}}. \tag{6}$$

Carbon Footprint. To estimate energy consumption during training and inference, we employ *Carbontracker* [1]. We report results in terms of carbon footprint rather than energy consumption directly, since energy is only problematic to the extent that it is generated from carbon-emitting sources. Were the grid entirely renewable, the same energy expenditure would carry negligible direct carbon emissions. In practice, however, the global energy mix remains far from carbon-neutral, and we therefore convert energy to carbon dioxide equivalent (CO_2eq) emissions using the global average carbon intensity for 2024 at 445 gCO_2/kWh [11]. Using a fixed carbon intensity ensures fair comparisons across models, independent of when or where they were trained.

4 Results

Predictive Performance. We first assess predictive performance at each step of a 50-step rollout across the four evaluation metrics described in Sec. 3. As seen in Fig. 3, all models perform well for the first few steps, after which performance declines at different rates. Among the unpadded convolutional models, UNet-CN outperforms UNet by a wide margin across all evaluation metrics, showing that the modern convolutional design substantially improves both accuracy and physical fidelity. Adding circular padding improves all four metrics for UNet, and improves or leaves roughly unchanged three of the four metrics for

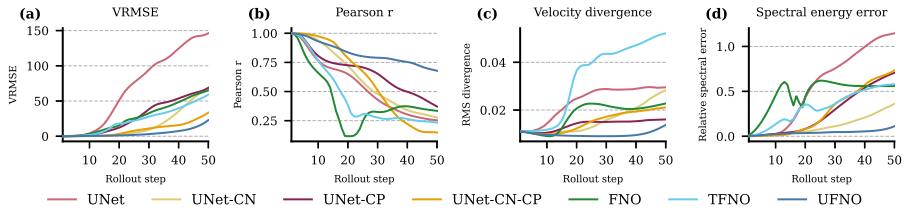


Fig. 3: Rollout performance over 50 steps, averaged over the 24 test trajectories. **a:** Variance-scaled root mean squared error (VRMSE) and **b:** Pearson correlation coefficient (Pearson r), each averaged over all four physical fields. **c:** Velocity divergence and **d:** spectral energy error, each computed on the velocity field only.

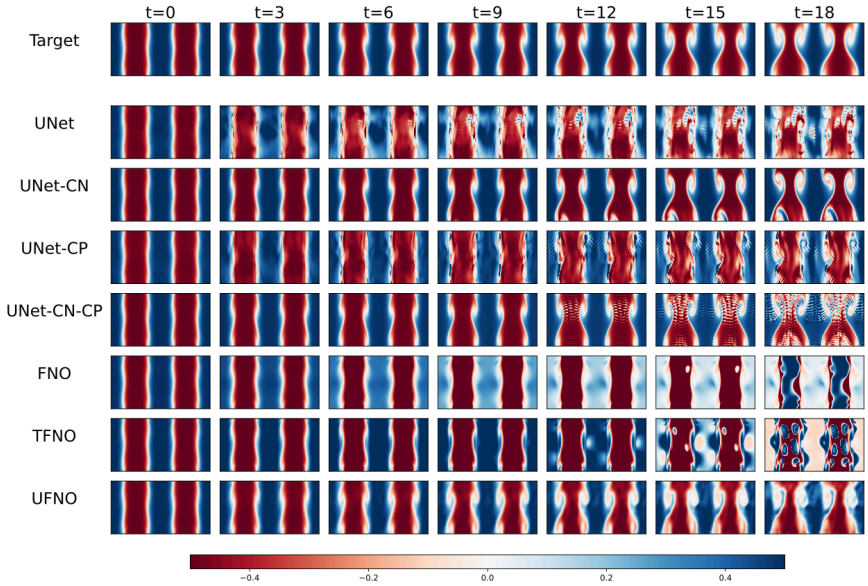


Fig. 4: Example rollout predictions for the tracer field. Target trajectory (top row) compared with model predictions shown every third step of a 20-step rollout.

UNet-CN (with spectral energy error being the exception). This suggests that encoding the domain's periodic boundary explicitly tends to help, though the benefit is not uniform across architectures and evaluation metrics. Among the neural operators, FNO and TFNO perform relatively poorly on all four metrics. In contrast, UFNO performs best across all four metrics, combining strong accuracy with low divergence and low spectral energy error. This suggests that the multi-scale U-shaped structure of UFNO, rather than the global receptive field alone, is what drives strong performance on this task. An example of a visual rollout of the tracer field for all models can be seen in Fig. 4.

Table 3: Carbon footprint during training and inference. CO_2eq emissions and run time for all models, measured with *Carbontracker* [1]. Inference results correspond to one-step forecasts over the full test set (4,704 samples) and 50-step rollouts over all test trajectories (24 samples). Inference costs are excluding the model-loading cost and are reported as mean plus/minus standard deviation over three runs. Inference times are reported as the mean over three runs.

	Training				Inference (one-step)		Inference (rollout)	
Model	**CO_2eq/epoch (g)**	**CO_2eq total (kg)**	**#Epochs**	**Time (h)**	**CO_2eq (g)**	**Time (s)**	**CO_2eq (g)**	**Time (s)**
UNet	63	2.72	43	12.7	3.59 ± 0.03	58	2.11 ± 0.02	44
UNet-CN	174	4.71	27	15.9	7.70 ± 0.05	113	2.66 ± 0.01	47
UNet-CP	109	3.81	35	14.2	3.99 ± 0.05	62	2.15 ± 0.00	44
UNet-CN-CP	131	5.89	45	20.9	4.80 ± 0.02	73	2.24 ± 0.04	46
FNO	67	0.80	12	4.0	7.80 ± 0.94	122	2.66 ± 0.29	50
TFNO	68	1.09	16	5.3	7.14 ± 0.12	107	2.81 ± 0.21	52
UFNO	37	1.12	30	3.6	8.94 ± 0.88	143	2.79 ± 0.05	48

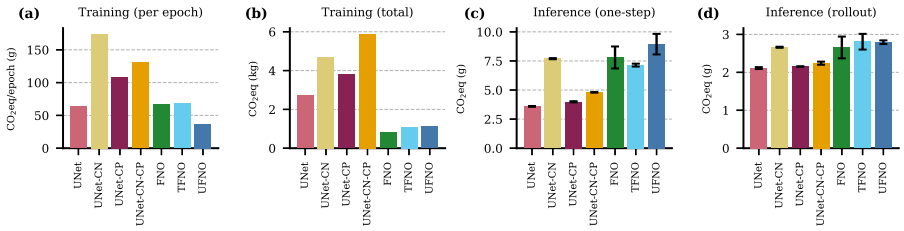


Fig. 5: Carbon footprint of training and inference. **a:** Training emissions per epoch (gCO_2eq). **b:** Total training emissions ($kgCO_2$eq). **c:** One-step inference emissions (gCO_2eq), summed over the full test set of 4,704 samples. **d:** Rollout inference emissions (gCO_2eq), summed over 24 autoregressive rollouts of 50 steps. Training emissions are from a single run per model, while inference emissions are from three repeated runs excluding model-loading cost.

Carbon Footprint. We next evaluate how carbon footprint varies with receptive field and periodicity assumption across training and inference (see Table 3 and Fig. 5). Among the unpadded convolutional models, per-epoch training cost differs sharply: UNet is among the cheapest models to train (63 gCO_2eq/epoch), while UNet-CN is the most carbon-intensive of all seven (174 gCO_2eq/epoch, Fig. 5a). This gap carries over to inference, where UNet remains the cheapest model at both one-step (3.59 gCO_2eq) and rollout (2.11 gCO_2eq) inference, while UNet-CN is among the costliest at one-step (7.70 gCO_2eq) and moderately expensive during rollout (2.66 gCO_2eq).

Adding circular padding affects the two backbones differently. For UNet, it raises per-epoch training cost substantially (63 to 109 gCO_2eq/epoch) while leaving inference cost almost unchanged (3.59 to 3.99 gCO_2eq one-step; 2.11 to 2.15 gCO_2eq rollout). For UNet-CN, adding circular padding instead lowers both carbon footprints, from 174 to 131 gCO_2eq/epoch during training, from 7.70 to 4.80 gCO_2eq at one-step inference, and from 2.66 to 2.24 gCO_2eq during rollout. The direction of the effect therefore depends on the backbone it is added to, rather than being a consistent property of the circular padding itself.

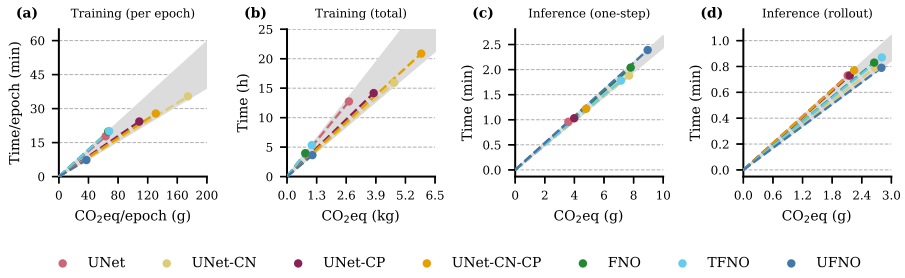


Fig. 6: Runtime vs. carbon footprint. **a:** Per-epoch training time vs. per-epoch training emissions (gCO_2eq). **b:** Total training time vs. total training emissions ($kgCO_2eq$). **c:** One-step inference time vs. one-step inference emissions (gCO_2eq). **d:** Rollout inference time vs. rollout inference emissions (gCO_2eq). For inference, each point marks a model's mean time and mean carbon footprint. The shaded region spans the ratios between the top and bottom dashed lines shown.

Among the neural operator variants, training and inference costs move in opposite directions compared to the convolutional models. FNO and TFNO train near to UNet's per-epoch cost (67 gCO_2eq/epoch and 68 gCO_2eq/epoch vs. 63 gCO_2eq/epoch), while UFNO trains at 55% of that cost (37 gCO_2eq/epoch), the lowest of all. At inference, however, all three FNO variants are among the most expensive: UFNO has the highest one-step cost of all seven models (8.94 gCO_2eq) despite having the lowest training cost, and UFNO together with TFNO has the highest rollout cost (2.79 gCO_2eq and 2.81 gCO_2eq).

UFNO is therefore a clear example of a model that is cheap to train but expensive to deploy, underscoring the need to consider resource costs across the full lifecycle rather than a single stage. A related pitfall is treating runtime as a proxy for energy consumption (or carbon footprint). Fig. 6 shows that runtime and carbon footprint are correlated but not strictly proportional across models for both training and inference stages. This is due to different hardware operations having distinct power profiles, meaning that a model that executes faster does not necessarily have a lower carbon cost [10].

5 Discussion

Our study explores how architectural biases, specifically a model's receptive field and periodicity assumption, are associated with the trade-off between efficacy and efficiency in spatio-temporal forecasting of incompressible shear flow. The answer is not straightforward as carbon trade-offs vary across the model lifecycle. However, this motivates the need to consider resource costs across the entire model lifecycle. Introducing an explicit periodicity assumption via circular padding is associated with better performance, but its effect on training carbon efficiency and one-step inference is itself model-dependent, increasing

both for UNet's padded counterpart (UNet-CP) and decreasing both for UNet-CN's padded counterpart (UNet-CN-CP). The neural operators (FNO, TFNO, UFNO) achieve the lowest total training footprints among all seven models, but this comes at the expense of higher inference costs relative to the convolutional models. For example, UFNO's one-step inference cost is nearly twice that of the best-performing convolutional model (UNet-CN-CP), despite UNet-CN-CP having a training cost per epoch more than three times that of UFNO. The lower total training cost of the neural operators is partly driven by fewer epochs to convergence under early stopping. As we use learning rates inherited from prior work rather than tuned per model, we cannot rule out that this untuned difference, rather than the architectural bias itself, drives the difference in convergence speed.

In terms of predictive performance, UFNO is the only model that performs well across both pointwise accuracy and physics-fidelity metrics for most of the 50-step rollout. Other models, such as UNet-CN, UNet-CP, and UNet-CN-CP, also do reasonably well for the first 10–15 steps. Since the models were only trained on one-step predictions and not on longer rollouts, it is, however, expected that they will eventually accumulate errors. The model that is most attractive on accuracy and physical fidelity, UFNO, also has the lowest training cost per epoch, though not the lowest inference cost. This means that no single model dominates efficacy, training cost, and inference cost simultaneously.

Our results further show that runtime is not a fully reliable proxy for carbon cost at either stage, though the gap is significantly larger for training than for both one-step and rollout inference. Taken together, these results show that model selection cannot be based on efficacy alone, nor on carbon cost at a single stage, nor on runtime as a stand-in for energy or carbon cost. Amortising development costs across the model lifecycle is one way to factor in the carbon costs [7].

Our results are specific to this PDE system, dataset subset, model family, and hardware setup, and should not be read as a general claim that architectural bias reduces carbon cost. We nonetheless believe that the broader question around incorporating inductive biases as a way to decrease data and compute requirements is worth raising since many scientific and engineering problems are governed by well-understood structures, and overlooking them is wasteful. However, inductive biases should not be incorporated at any cost, but only where they provides clear benefits in terms of predictive performance or resource efficiency. While our work centres on architectural biases, the same logic extends to other forms of prior knowledge: any regularising bias that constrains the learning task appropriately can potentially offer trade-offs between efficacy and efficiency.

Ultimately, our study argues for a shift in mindset. Progress in ML should not be equated with ever-larger datasets, models, and compute budgets, but rather evaluated in terms of the value gained relative to the cost incurred [31]. Learning everything from data while ignoring domain knowledge reflects an abundance mindset, in which resources are treated as unlimited. A more sustainable alternative is to exploit what we already know by embedding this knowledge into models. This work offers a step towards understanding how architectural biases relate to the efficacy-efficiency trade-offs, and argues for evaluating these explicitly.

Limitations. Model configurations, including learning rates, follow prior work tuned on several PDE datasets, including the shear-flow dataset used here. Notably, the learning rate differs by a factor of two between the UNet and FNO families (5×10^{-4} vs. 10^{-3}). This is a plausible confound for total training emissions, since these depend on the number of epochs to early stopping, which varies by roughly a factor of four across models (12–45). Training carbon and epoch counts are also reported from a single run per model, so we cannot rule out that run-to-run variance in convergence contributes to the reported differences. We restricted our study to one PDE system, two model families, and a single GPU, and do not claim these findings generalise beyond this setting.

6 Conclusions

The primary message in this work, backed by empirical evidence, is to increase resource-awareness when designing ML experiments. While this can be accomplished in several ways [3], we focus on the role of inductive biases and illustrate the different decisions practitioners have to consider. Using spatio-temporal forecasting of incompressible shear flow, we show how architectural biases are associated with the trade-off between predictive performance and carbon footprint.

This work advances a perspective grounded in resource-awareness and calls for a shift in mindset. As a community, we should discourage the pursuit of marginal improvements in performance that come from disproportionately higher resource costs. With our work, we have made the case to encourage resource-awareness by utilising inductive biases that encode existing domain knowledge.

Acknowledgements

SW and RS acknowledge funding received from Independent Research Fund Denmark (DFF) under grant agreement number 4307-00143B. JHC acknowledges funding by DFF through the project Arctic Push, grant number 4258-00025B. RS also acknowledges funding received under European Unions Horizon Europe Research and Innovation Action programme under grant agreements No. 101070284, No. 101070408 and No. 101189771.

The authors thank Erik B Dam, Pedram Bakhtiarifard, Yijun Bian and others from the Machine Learning Section at UCPH for constructive feedback on the manuscript. The authors thank members of SAINTS Lab for useful discussions throughout.

References

1. Anthony, L.F.W., Kanding, B., Selvan, R.: Carbontracker: Tracking and predicting the carbon footprint of training deep learning models (2020), https://arxiv.org/abs/2007.03051
2. Bakhtiarifard, P., Igel, C., Selvan, R.: EC-NAS: Energy consumption aware tabular benchmarks for neural architecture search. In: ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). p. 56605664. IEEE (Apr 2024). https://doi.org/10.1109/icassp48485.2024.10448303, http://dx.doi.org/10.1109/ICASSP48485.2024.10448303
3. Bartoldson, B.R., Kailkhura, B., Blalock, D.: Compute-efficient deep learning: Algorithmic trends and opportunities. Journal of Machine Learning Research **24**(122), 1–77 (2023), https://jmlr.org/papers/v24/22-1208.html
4. Brehmer, J., Behrends, S., de Haan, P., Cohen, T.: Does equivariance matter at scale? (2024), https://arxiv.org/abs/2410.23179
5. Burns, K.J., Vasil, G.M., Oishi, J.S., Lecoanet, D., Brown, B.P.: Dedalus: A flexible framework for numerical simulations with spectral methods. Physical Review Research **2**(2), 023068 (2020). https://doi.org/10.1103/PhysRevResearch.2.023068
6. Chen, Z., Liu, Y., Sun, H.: Physics-informed learning of governing equations from scarce data. Nature communications **12**(1), 6136 (2021), https://www.nature.com/articles/s41467-021-26434-1
7. Cottier, B., Rahman, R., Fattorini, L., Maslej, N., Besiroglu, T., Owen, D.: The rising costs of training frontier ai models (2024), https://arxiv.org/abs/2405.21015
8. Dutta, S., Innan, N., Yahia, S.B., Shafique, M.: AQ-PINNs: Attention-enhanced quantum physics-informed neural networks for carbon-efficient climate modeling (2024), http://arxiv.org/abs/2409.01626
9. Evchenko, M., Vanschoren, J., Hoos, H.H., Schoenauer, M., Sebag, M.: Frugal machine learning (2021), https://arxiv.org/abs/2111.03731
10. Henderson, P., Hu, J., Romoff, J., Brunskill, E., Jurafsky, D., Pineau, J.: Towards the systematic reporting of the energy and carbon footprints of machine learning (2022), https://arxiv.org/abs/2002.05651
11. IEA: Electricity 2025. https://www.iea.org/reports/electricity-2025 (2025), accessed: 2025-09-20
12. Jahani-nasab, M., Bijarchi, M.A.: Enhancing convergence speed with feature enforcing physics-informed neural networks using boundary conditions as prior knowledge. Scientific Reports **14**, 23836 (2024). https://doi.org/10.1038/s41598-024-74711-y, https://doi.org/10.1038/s41598-024-74711-y
13. Kapoor, T., Chandra, A., Stamou, A., Roberts, S.J.: Beyond accuracy: Ecol2 metric for sustainable neural pde solvers (2025), https://arxiv.org/abs/2505.12556
14. Khoa, N.: Tutorials for physics-informed neural networks (pinns) (2022), https://github.com/nguyenkhoa0209/pinns_tutorial, accessed: 2024-07-12
15. Kohavi, R., Wolpert, D.H., et al.: Bias plus variance decomposition for zero-one loss functions. In: Proceedings of the International Conference on Machine Learning (ICML). vol. 96, pp. 275–283 (1996), https://dl.acm.org/doi/10.5555/3091696.3091730
16. Kossaifi, J., Kovachki, N., Azizzadenesheli, K., Anandkumar, A.: Multi-grid tensorized Fourier neural operator for high-resolution pdes (2023), https://arxiv.org/abs/2310.00120

17. Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., Anandkumar, A.: Fourier neural operator for parametric partial differential equations. In: International Conference on Learning Representations (ICLR) (2021), https://openreview.net/pdf/53c47f849d1cd4d21b865caf7d774e07a5c42aa4.pdf
18. Liu, Z., Mao, H., Wu, C.Y., Feichtenhofer, C., Darrell, T., Xie, S.: A convnet for the 2020s. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 11976–11986 (2022). https://doi.org/10.1109/CVPR52688.2022.01167
19. Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. In: International Conference on Learning Representations (2019), https://openreview.net/forum?id=Bkg6RiCqY7
20. Luccioni, A.S., Viguier, S., Ligozat, A.L.: Estimating the carbon footprint of BLOOM, a 176b parameter language model. Journal of machine learning research **24**(253), 1–15 (2023), https://www.jmlr.org/papers/v24/23-0069.html
21. Moro, G., Ruggazzi, L., Valgimigli, L.: Carburacy: Summarization models tuning and comparison in eco-sustainable regimes with a novel carbon-aware accuracy. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 37, pp. 14417–14425 (2023). https://doi.org/10.1609/aaai.v37i12.26686
22. Ohana, R., McCabe, M., Meyer, L., Morel, R., Agocs, F.J., Beneitez, M., Berger, M., Burkhart, B., Dalziel, S., Fielding, D., et al.: The Well: A large-scale collection of diverse physics simulations for machine learning. Dataset (2024). https://doi.org/10.17863/CAM.113689, https://doi.org/10.17863/CAM.113689, accepted at NeurIPS 2024 Datasets and Benchmarks
23. Patra, S., Panda, S., Parida, B.K., Arya, M., Jacobs, K., Bondar, D.I., Sen, A.: Physics informed Kolmogorov-Arnold neural networks for dynamical analysis via efficient-kan and wav-kan (2024), https://arxiv.org/abs/2407.18373
24. Psaros, A.F., Pestourie, R., Rakich, P.T., Johnson, S.G., Dey, P., Das, P., Moucer, J., et al.: Physics-enhanced deep surrogates for partial differential equations. Nature Machine Intelligence **5**, 1458–1468 (2023). https://doi.org/10.1038/s42256-023-00761-y
25. Rahman, M.A., Ross, Z.E., Azizzadenesheli, K.: U-NO: U-shaped neural operators (2023), https://arxiv.org/abs/2204.11127
26. Ronneberger, O., Fischer, P., Brox, T.: U-net: Convolutional networks for biomedical image segmentation. In: Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015. LNCS, vol. 9351, pp. 234–241. Springer (2015). https://doi.org/10.1007/978-3-319-24574-4_28
27. Sevilla, J., Heim, L., Ho, A., Besiroglu, T., Hobbhahn, M., Villalobos, P.: Compute trends across three eras of machine learning. In: 2022 International Joint Conference on Neural Networks (IJCNN). p. 18. IEEE (Jul 2022). https://doi.org/10.1109/ijcnn55064.2022.9891914, http://dx.doi.org/10.1109/IJCNN55064.2022.9891914
28. Strubell, E., Ganesh, A., McCallum, A.: Energy and policy considerations for deep learning in NLP. In: Korhonen, A., Traum, D., Màrquez, L. (eds.) Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics. pp. 3645–3650. Association for Computational Linguistics, Florence, Italy (Jul 2019). https://doi.org/10.18653/v1/P19-1355, https://aclanthology.org/P19-1355
29. Tan, M., Le, Q.: EfficientNet: Rethinking model scaling for convolutional neural networks. In: International conference on machine learning. pp. 6105–6114. PMLR (2019), https://proceedings.mlr.press/v97/tan19a.html

30. Wen, G., Li, Z., Azizzadenesheli, K., Anandkumar, A., Benson, S.M.: U-Fnoan enhanced Fourier neural operator-based deep-learning model for multiphase flow. Advances in Water Resources **163**, 104180 (2022)
31. Wilson, S.N., Mair, S., Okinyi, M., Dam, E.B., Koch, J., Selvan, R.: How Hyper-Datafication Impacts the Sustainability Costs in Frontier AI. In: Proceedings of the 2026 ACM Conference on Fairness, Accountability, and Transparency (June 2026), https://dl.acm.org/doi/10.1145/3805689.3812393
32. Zhao, Y., Liu, Y., Jiang, B., Guo, T.: CE-NAS: An end-to-end carbon-efficient neural architecture search framework. Advances in Neural Information Processing Systems **37**, 82673–82704 (2024), https://dl.acm.org/doi/10.5555/3737916.3740544
33. Zhong, Y.D., Dey, B., Chakraborty, A.: Benchmarking energy-conserving neural networks for learning dynamics from data. In: Jadbabaie, A., Lygeros, J., Pappas, G.J., Parrilo, P.A., Recht, B., Tomlin, C.J., Zeilinger, M.N. (eds.) Proceedings of the 3rd Conference on Learning for Dynamics and Control. Proceedings of Machine Learning Research, vol. 144, pp. 1218–1229. PMLR (07 – 08 June 2021), https://proceedings.mlr.press/v144/zhong21a.html